

Exercises 4: Perceptron

Exercise 1

Consider two perceptrons with the following weights and biases:

1. $w_1 = (0.5, 1.5)$, $b_1 = -0.5$
2. $w_2 = (-2, 0.5)$, $b_2 = 1$

Given the input $x = (2, -1)$, calculate both neurons' output with the following activation functions: a.) Heaviside b.) ReLU c.) Sigmoid d.) Tanh

Exercise 2

Solve the two simple classification problems shown in Figure 1 by drawing a decision boundary between black points (classified as 1) and white points (classified as 0). Find weight and bias values that result in single-neuron perceptrons with the chosen decision boundaries.

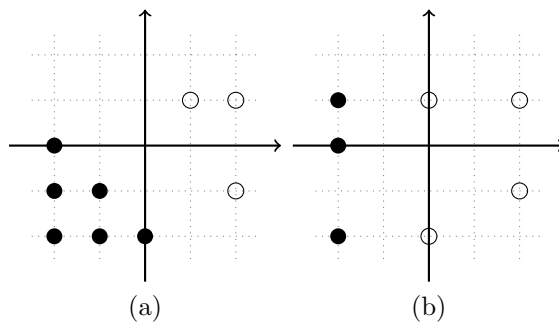


Figure 1: Binary data set

Exercise 3

You are given the following dataset with four data points, each consisting of two input features (x_1, x_2) and a corresponding label y :

$$\begin{aligned}
(x_1, x_2) &= (1, 1), y = 1 \\
(x_1, x_2) &= (1, -1), y = -1 \\
(x_1, x_2) &= (-1, 1), y = -1 \\
(x_1, x_2) &= (-1, -1), y = -1
\end{aligned}$$

Initialize weights of a perceptron as

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

Apply one full iteration of the perceptron weight update for every data point using a learning rate of $\eta = 0.1$ and the following learning rule:

$$\begin{aligned}
w_j &\leftarrow w_j + \eta \cdot (y_p - O_p) \cdot x_{pj} \\
b &\leftarrow b + \eta \cdot (y_p - O_p)
\end{aligned}$$

where O_p is the perceptron output:

$$O_p = \text{sgn}(w_1 x_1 + w_2 x_2 + b)$$

with

$$\text{sgn}(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0. \end{cases}$$

Exercise 4

Consider a neural network with the following architecture:

- 2 input features (x_1, x_2)
- 2 neurons in the hidden layer, with ReLU activation
- 1 neuron in the output layer, with sigmoid activation
- The loss function is Mean Squared Error (MSE)

Training Data:

x_1	x_2	Expected Output y
0.3	0.7	0.6

Initial Parameters:

$$\mathbf{W}^1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad \mathbf{W}^2 = \begin{bmatrix} 0.5 & 0.6 \end{bmatrix}, \quad \mathbf{b}^1 = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}, \quad b^2 = 0.1$$

Where w_{ij}^k is the weight for node i in layer k for incoming input j .

Learning Rate: $\eta = 0.1$

Perform one iteration of the back-propagation algorithm by completing the following steps:

1. Perform the forward pass and compute the output of the network.
2. Calculate the loss.
3. Compute the gradients for the weights $\mathbf{W}_1, \mathbf{W}_2$ and biases $\mathbf{b}_1, b_2$ using backpropagation.
4. Update the weights and biases.

Hint: The derivatives of the sigmoid and ReLU functions are:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)),$$

$$\text{ReLU}'(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

where z is the net output of a layer.

Exercise 5

Given a perceptron with two boolean inputs x_1 and x_2 , where both inputs can take values from the set $\{0, 1\}$, and a step function for the prediction $\hat{y}$:

$$\hat{y} = \begin{cases} 1 & \text{if } W \cdot x + b > 0 \\ 0 & \text{if } W \cdot x + b \leq 0 \end{cases}$$

we can construct a perceptron for a boolean operator AND (see Figure 2).

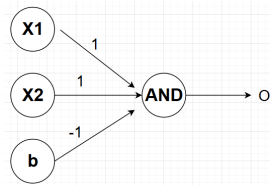


Figure 2: And gate perceptron

Draw a perceptron that can compute the following Boolean operations:

- a. **OR** (logical disjunction): $x_1 \vee x_2$
- b. **NOT** (logical negation): $\neg x$
- c. **NAND** (negated AND): $\overline{x_1 \wedge x_2}$
- d. **NOR** (negated OR): $\overline{x_1 \vee x_2}$.

- e. As discussed in the lecture, the **XOR** operation is not linearly separable. Draw a diagram of a multi-layer perceptron network that implements the XOR operator.

Hint: A standard Boolean representation for the XOR function is:

$$x_1 \oplus x_2 = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2).$$

This can be equivalently expressed as:

$$x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \overline{(x_1 \wedge x_2)}.$$

You can thus combine previously created perceptrons (AND, OR, NAND) to compute this Boolean expression.